From Pillars to Transformers: Two Fusion Paradigms Based on PointPillars for 3D Object Detection

A Project Report
Presented to
The Faculty of the Department of Engineering
San José State University
In Partial Fulfillment of the Requirements for the Degree
Master of Science

by
Le Shen, Pavithra Mahadevappa Girithimmappa, Sanghee Suh and Xiaotong Wu
April 2026

ABSTRACT

LiDAR–camera fusion has emerged as an effective approach for improving 3D object detection in autonomous driving by combining accurate geometric information with rich visual semantics. This work presents two complementary fusion architectures built upon the PointPillars backbone. The first, FusionPointPillars, is a lightweight multimodal framework that performs efficient pillar-wise fusion by projecting LiDAR pillars onto the image plane and aggregating multi-scale image features extracted using a ResNet-50 backbone and Feature Pyramid Network (FPN). The fused features are integrated into a bird's-eye-view (BEV) representation while preserving the efficiency of the original pipeline. Three fusion strategies Single-Scale Fusion (SSF), Global Multi-Scale Fusion (GMF), and Adaptive Pillar Fusion (APF) are evaluated on the KITTI benchmark, showing improved performance for challenging classes such as pedestrians and cyclists by GMF and APF. We also benchmarked inference efficiency on a local machine equipped with an NVIDIA GeForce RTX 3060 GPU (12 GB), an Intel® Core™ i5-10400 CPU, and 32 GB RAM. The LiDAR-only PointPillars baseline achieves 37.50 ms latency per frame, while the fusion variants achieve 84.07 ms for SSF, 82.95 ms for GMF, and 83.36 ms for APF. These results indicate that the added cost of multimodal fusion is primarily due to image feature extraction, while the different multi-scale fusion strategies introduce only minor additional overhead. The added latency still remains within a practical near-real-time range.

The second architecture, BEVFormerFusion, extends a BEVFormer-based camera detector by incorporating LiDAR features through decoder-side fusion and, in the full Enc_Dec variant, through additional encoder-side cross-modal attention. Experiments on the nuScenes validation set demonstrate that Enc_Dec improves the camera-only baseline from 0.219 to 0.255 NDS and from 0.201 to 0.251 mAP. We peformed the latency tests for BEVFormerFusion with using the same experimental setup as for FusionPointPillars. The Baseline, Decoder-only (V3), and Enc_Dec variants achieved 259.32 ms, 269.31 ms, and 431.47 ms average latency per sample, respectively. These results show that LiDAR-camera fusion is feasible in both lightweight and transformer-based designs, while also highlighting a clear trade-off between detection accuracy and runtime cost.

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to our advisor, Ahmed Hambaba, Ph.D., Professor and MSE Advisor, for his invaluable guidance, technical expertise, and continuous support throughout the duration of this project. His insights and encouragement were instrumental in the successful completion of this research.

Finally, we thank our families and peers for their patience and support during this academic journey.

Contents

1 INTRODUCTION

Accurate 3D perception is essential for autonomous vehicles navigating complex and dynamic environments with moving and stationary objects like vehicles, pedestrians, and traffic cones. Modern autonomous driving systems rely heavily on multi-modal systems, most commonly LiDAR and multi-view cameras, to understand the surrounding scene more thoroughly[**?**], [**?**], [**?**].

LiDAR sensors provide precise geometric measurements with accurate depth information, making them well suited for spatial localization. Nevertheless, they exhibit inherent sparsity and can be increasingly unreliable when detecting distant or small objects, such as pedestrians and cyclists. In contrast, camera sensors offer dense appearance and semantic information at high spatial resolution but lack direct depth measurements, leading to ambiguity in 3D localization. These complementary characteristics naturally motivate multimodal fusion strategies.

Despite rapid progress in individual LiDAR-based and camera-based BEV methods, multi-sensor fusion in BEV space remains a research challenge. Enhanced object detection by some approaches such as MVX-Net [**?**], Multimodal PointPillars [**?**], and BEVFusion [**?**] illustrate the value of combining geometric information from LiDAR and semantic information from camera. Most previous fusion models rely on voxel-based or point-level operations that are expensive to compute or difficult to scale. A computationally more efficient fusion approach is needed to integrate LiDAR geometry and camera semantics directly in BEV space.

Research on 3D object detection has recently shifted toward representing sensor data in Bird's-Eye View (BEV) space, a top-down representation that enables spatially consistent reasoning and simplifies multi-sensor alignment. Transformer-based architectures such as BEVFormer have demonstrated strong capabilities in generating BEV features from multi-view cameras by leveraging spatial cross-attention for image-to-BEV projection and temporal self-attention for multi-frame aggregation [**?**].

Recently, Li et al. [**?**] introduced a multi-modal BEVFormer model which fuses LiDAR features and camera image features on BEV via spatio-temporal transformers. They used a VoxelNet model to produce LiDAR features and their modified BEVFormer to fuse the generated LiDAR features with camera image features. Their approach achieved the state-of-the-art 74.1% in terms of NDS metric on the nuScenes test set. Even though their model is very accurate, it would be difficult to overcome its slow inference speed because of the heavy transformer and VoxelNet requiring 3D convolution.

As part of the ENGR 297D class project [**?**], we conducted an initial investigation about fusion of camera and LiDAR data. The goal of this study was to explore effi-

cient methods for integrating geometric and semantic information from these two sensor modalities for improving perception accuracy and processing speed. Our research was motivated by existing works, specifically the MVX-Net (VoxelFusion) model [**?**], which demonstrated a successful approach to feature fusion. In MVX-Net, voxel features are derived from LiDAR data using Voxel Feature Encoding (VFE), while camera image features are extracted with a convolutional neural network (CNN) backbone. These sets of features are then fused to enhance the model's understanding of the environment. Inspired by the MVX-Net approach, we developed our own model, FusionPointPillars, designed to integrate LiDAR and camera image features on a bird's-eye view (BEV) representation. The FusionPointPillars model aims to achieve faster inference than the MVX-Net (VoxelFusion) method, while maintaining high prediction accuracy. This efficiency is anticipated through a streamlined fusion process, leveraging the complementary strengths of both sensor types.

In this project, we introduce two PointPillars-based fusion directions. The first, **FusionPointPillars**, is a lightweight LiDAR–camera fusion model that combines image features with pillar embeddings before BEV processing, enhancing object recognition without sacrificing efficiency. The second is a **spatiotemporal BEV fusion framework** that merges relatively fast PointPillars-based LiDAR extraction and BEVFormer-based multi-view camera encoding, employing cross-attention to align geometric and semantic data. This unified architecture improves perception accuracy, robustness, and efficiency for autonomous driving.

2 RELATED WORK

2.1 CAMERA-ONLY 3D OBJECT DETECTION

The widespread use and affordability of RGB cameras have led to increased research in monocular and multi-view 3D object detection. Early monocular methods like Mono3D and Deep3DBox used geometric assumptions, depth information, or object shape limitations to estimate 3D bounding boxes [**?**], [**?**]. Later works leveraged learned depth estimation, as in Pseudo-LiDAR, which transforms predicted depth maps into pseudo point clouds to emulate LiDAR detection pipelines [**?**].

Transformer-based BEV detection models have recently advanced camera-only object detection. DETR3D introduces object queries that attend directly to multi-view image features through 3D-to-2D projection, enabling end-to-end detection without dense depth maps [**?**]. PETR and PETRv2 represent image features as learnable 3D position-aware queries and decode them into a unified BEV representation using transformer encoders [**?**], [**?**]. BEVFormer (2022) further improves camera-only BEV reasoning through spatiotemporal transformers, combining deformable attention across multiple camera views and past BEV features [**?**].

2.2 POINT-BASED AND VOXEL-BASED LIDAR DETECTION

Early deep networks for point clouds focused on utilizing unordered point sets directly. PointNet introduced a permutation-invariant architecture that applies shared MLPs and symmetric pooling to learn global shape information [**?**]. While effective, PointNet lacks local neighborhood modeling. Voxel-based approaches address this limitation by converting point clouds to a regular 3D grid. VoxelNet uses Voxel Feature Encoding (VFE) to aggregate local point geometry within each voxel through stacked PointNet-style layers, then applies 3D convolutions for context reasoning [**?**]. These methods achieve strong 3D detection accuracy but incur substantial computational and memory overhead due to dense 3D convolutions.

MVX-Net extended VoxelNet to the multimodal setting via PointFusion and VoxelFusion, appending image features to raw points or voxel features before 3D convolutions [**?**]. This early fusion strategy improves performance when LiDAR is sparse or ambiguous. However, MVX-Net still inherits the computational burden of 3D CNNs and requires separate 2D feature extraction, making real-time deployment challenging.

2.3 PILLAR-BASED LIDAR DETECTION

PointPillars proposed a lightweight LiDAR detection pipeline by collapsing the vertical axis and encoding points as features in vertical pillars rather than voxels [**?**]. A Pillar Feature Network (PFN) converts each pillar into a fixed-length embedding, which is scattered into a BEV pseudo-image and processed by a 2D CNN. This dramatically improves inference speed while maintaining strong 3D detection accuracy, establishing PointPillars as an efficient baseline for resource-constrained settings.

Multimodal PointPillars (M-PP) [**?**] extends PointPillars by incorporating early fusion between LiDAR and image features. M-PP first performs pillarization and learns per-pillar features, then projects the LiDAR points into the image plane and indexes corresponding 2D CNN activations. These image features, which are typically extracted from a mid-level backbone layer, are fused with point features via point-wise concatenation before PFN aggregation. The fused pillar features are then scattered back into a BEV pseudo-image and processed by the standard PointPillars backbone and detection head. This early fusion strategy improves detection performance, particularly for small objects such as pedestrians, by enriching pillar embeddings with visual semantic cues while retaining the real-time efficiency of PointPillars.

2.4 SEQUENTIAL AND POINT-LEVEL FUSION

Sequential fusion methods enrich LiDAR point clouds using pixel-level image features. PointPainting projects LiDAR points into a semantic segmentation output from a 2D network and appends class distributions to each point [**?**]. This augmented point cloud can be consumed by any LiDAR detector, improving performance across multiple backbones. While simple and effective, PointPainting depends heavily on the segmentation model quality and requires a two-stage pipeline, decoupling image understanding from 3D detection and preventing end-to-end optimization.

2.5 UNIFIED BEV FUSION AND TRANSFORMER-BASED MULTIMODAL ARCHITECTURES

Recent work has shifted toward fusing image and LiDAR features in a shared BEV representation. BEVFusion introduces an efficient camera-to-BEV transformation module and merges camera and LiDAR BEV features for multi-sensor perception tasks, achieving strong results on nuScenes [**?**]. BEVFormer (2022) models spatiotemporal relationships exclusively from camera images using cross-attention between BEV queries and multi-view image features [**?**]. More recent transformer-based multimodal variants

(BEVFormer+, 2025) extend this framework to LiDAR–camera fusion, enabling BEV queries to interact with point cloud features as well as multi-view images [**?**]. These models have produced state-of-the-art results.

3 METHOD

3.1 IMPLEMENTATION OF FUSIONPOINTPILLARS

3.1.1 OVERVIEW

We implemented FusionPointPillars using MMDetection3D [**?**], an open-source PyTorch-based toolbox that provides modular and widely adopted implementations for LiDAR- and camera-based 3D object detection. The overall pipeline consists of four main stages: (1) pillar feature encoding from LiDAR point clouds, (2) extraction of image features using a 2D backbone with optional feature pyramid networks (FPN) [**?**], (3) pillar-wise projection and ROI-based fusion of image features, and (4) BEV feature learning and 3D bounding box prediction. An overview of the system architecture is illustrated in Fig. 1.

As our baseline, we follow the standard PointPillars configuration [**?**], including pillarization of point clouds, Pillar Feature Network (PFN) encoding, BEV feature scattering, and a single-stage anchor-based detection head. Specifically, we adopt the BEV backbone and SSD-style 3D detection head design introduced in SECOND [**?**], which has become a common choice for efficient LiDAR-based perception.

Building on this baseline, FusionPointPillars introduces a pillar-wise LiDAR–camera fusion module while keeping all other architectural components, training procedures, and evaluation settings unchanged. This design choice enables a controlled comparison and ensures that observed performance gains can be attributed directly to the proposed fusion strategy.

3.1.2 PILLAR FEATURE ENCODING

The LiDAR point cloud is discretized into an evenly spaced grid on the ground plane, forming a set of vertical columns referred to as pillars. Each non-empty pillar aggregates all LiDAR points whose (x, y) coordinates fall within the same pillar. A LiDAR point is represented as

$$\mathbf{l}_p = (x_p, y_p, z_p, r), \tag{1}$$

where (x_p, y_p, z_p) denote the 3D coordinates of an individual LiDAR point in the LiDAR frame and r denotes the reflectance (intensity) returned by the sensor.

For each pillar, points are augmented with offsets relative to both the arithmetic mean of points within the pillar and the geometric center of the pillar in the BEV plane. This augmentation yields a 9-dimensional feature vector per point.

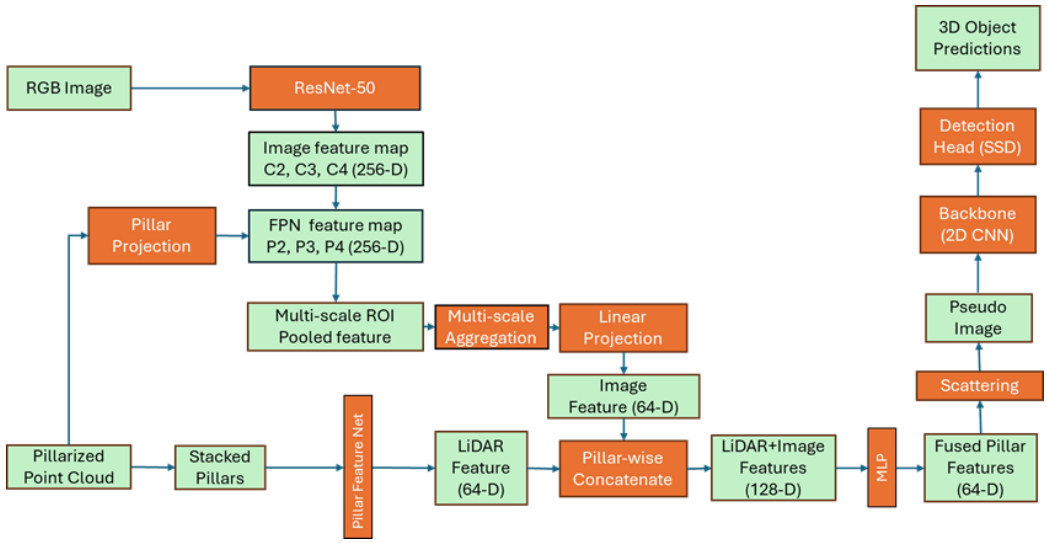


Fig. 1. Overview of FusionPointPillars architecture. The LiDAR point cloud is first pillarized and encoded into 64-dimensional pillar features using a Pillar Feature Network (PFN). Each pillar center is projected onto the image plane, where local 3×3 regions are sampled from multi-scale image feature maps produced by a ResNet-50 backbone and feature pyramid network (FPN). The resulting image features are aggregated and projected to match the pillar feature dimension, then fused with the LiDAR pillar features via a lightweight MLP. The fused pillar features are passed through a lightweight MLP and then scattered into a BEV pseudo-image and processed by a BEV backbone and single-stage detection head to produce final 3D bounding box predictions.

A PointNet-style Pillar Feature Network (PFN) applies a shared multi-layer perceptron (MLP) to all points within a pillar, followed by max pooling to obtain a fixed-length pillar embedding:

$$\mathbf{p}_i \in \mathbb{R}^{C_{\text{pillar}}}, \tag{2}$$

where $C_{\text{pillar}} = 64$ in our implementation. After pillarization and PFN encoding, the resulting tensor is

$$\texttt{pillar_features} \in \mathbb{R}^{N_{\text{pillars}} \times C_{\text{pillar}}}, \tag{3}$$

with corresponding discrete pillar coordinates

$$\texttt{pillar_coords} = [b, z_i, y_i, x_i], \tag{4}$$

where b denotes the batch index. We distinguish point-level coordinates (x_p, y_p, z_p) from pillar-level quantities (x_i, y_i, z_i) used in subsequent sections to denote pillar centers. The ordering (z_i, y_i, x_i) follows the tensor indexing convention used by MMDetection3D and PointPillars, where discrete pillar coordinates are stored as $[b, z, y, x]$ to match the underlying BEV feature layout.

Up to this stage, the architecture is identical to the LiDAR-only PointPillars back-

bone.

3.1.3 IMAGE FEATURE EXTRACTION

An RGB image is processed by a convolutional backbone network, specifically ResNet–50 [**?**], which produces a hierarchy of intermediate feature maps at increasing depths. We denote the output feature maps from the second, third, and fourth backbone stages as

$$\{C_2, C_3, C_4\}, \tag{5}$$

where each stage captures progressively higher-level semantic information at lower spatial resolutions. Concretely, C_2, C_3, and C_4 have spatial resolutions of $\frac{1}{4}$, $\frac{1}{8}$, and $\frac{1}{16}$ of the input image, respectively, with corresponding channel dimensions of 256, 512, and 1024.

With a Feature Pyramid Network (FPN) [**?**] employed, these backbone features are further transformed into a set of multi-scale feature maps

$$\{P_2, P_3, P_4\}, \tag{6}$$

where each pyramid level P_ℓ has a unified channel dimension of $C_{\text{img}} = 256$. The spatial resolutions of P_2, P_3, and P_4 correspond to $\frac{1}{4}$, $\frac{1}{8}$, and $\frac{1}{16}$ of the input image, respectively, matching the resolutions of their associated backbone features. By combining high-resolution spatial detail from shallow layers with high-level semantic context from deeper layers, the FPN provides a rich multi-scale representation of image information that complements LiDAR geometry for downstream fusion.

3.1.4 FUSION STRATEGIES FOR MULTI-SCALE IMAGE AGGREGATION

To systematically study the role of multi-scale image fusion, we investigate three fusion strategies of increasing adaptivity and complexity: Single-Scale Fusion (SSF), Global Multi-Scale Fusion (GMF), and Adaptive Pillar Fusion (APF).

Let $\mathbf{p}_i \in \mathbb{R}^{C_{\text{pillar}}}$ denote the LiDAR pillar feature of pillar i, and let $\{\mathbf{f}_i^{(l)}\}_{l=1}^{L}$ denote the ROI-pooled image features extracted from FPN levels P_l.

1) Single-Scale Fusion (SSF)

In the simplest setting, image information is extracted from a single feature map level, specifically P_4, which provides strong semantic representation. The fused image feature is defined as

$$\mathbf{f}_i^{\text{img}} = \mathbf{f}_i^{(4)}. \tag{7}$$

2) Global Multi-Scale Fusion (GMF)

To incorporate semantic information from multiple pyramid levels, we introduce learnable global fusion weights

$$\alpha = \{\alpha_1, \dots, \alpha_L\}, \tag{8}$$

which are normalized via a softmax operation such that

$$\sum_{l=1}^{L} \alpha_l = 1. \tag{9}$$

The aggregated image feature is then computed as

$$\mathbf{f}_i^{\text{img}} = \sum_{l=1}^{L} \alpha_l \, \mathbf{f}_i^{(l)}. \tag{10}$$

In this formulation, the weights α_l are shared across all pillars and images. The model learns a dataset-level preference over feature scales, but the weighting does not depend on individual object geometry or local scene context. This corresponds to a global multi-scale aggregation strategy.

3) Adaptive Pillar Fusion (APF)

We now introduce Adaptive Pillar Fusion (APF), which extends GMF by making the fusion weights dependent on each individual pillar feature. Instead of using fixed global weights, we predict pillar-specific fusion weights

$$\alpha_i = \{\alpha_{i,1}, \alpha_{i,2}, \dots, \alpha_{i,L}\}. \tag{11}$$

These weights are computed as a function of the LiDAR pillar feature:

$$\alpha_i = \text{softmax}\big(g(\mathbf{p}_i)\big), \tag{12}$$

where $g(\cdot)$ is a lightweight multilayer perceptron. The softmax normalization ensures

$$\sum_{l=1}^{L} \alpha_{i,l} = 1. \tag{13}$$

The aggregated image feature for pillar i becomes

$$\mathbf{f}_i^{\text{img}} = \sum_{l=1}^{L} \alpha_{i,l} \, \mathbf{f}_i^{(l)}. \tag{14}$$

Unlike GMF, the weights $\alpha_{i,l}$ vary across pillars and are conditioned on local LiDAR geometry. This enables the network to emphasize different semantic scales depending on object size, density, and spatial configuration.

3.1.5 FEATURE PROJECTION AND FUSION

After multi-scale aggregation via SSF, GMF, or APF, the image feature is projected to match the pillar feature dimension:

$$\mathbf{f}'_i = W_{\text{img}}\, \mathbf{f}_i^{\text{img}}. \tag{15}$$

The projected image feature is then concatenated with the LiDAR feature:

$$\mathbf{h}_i = [\, \mathbf{p}_i \,\|\, \mathbf{f}'_i \,]. \tag{16}$$

A lightweight MLP produces the final fused representation:

$$\mathbf{p}_i^{\text{fused}} = \phi(W_{\text{fuse}}\, \mathbf{h}_i)\,, \tag{17}$$

where $\phi(\cdot)$ denotes Batch Normalization followed by ReLU.

3.1.6 BEV BACKBONE AND DETECTION HEAD

After fusion, the enhanced pillar features are scattered into a 2D bird's-eye-view (BEV) pseudo-image that spans the discretized ground plane used during pillarization. Each BEV cell stores a fused feature vector of dimension 64, enabling efficient processing with standard 2D convolutions while preserving metric scene geometry.

The BEV pseudo-image is processed by the same backbone architecture as in PointPillars. Specifically, we employ a shallow 2D convolutional network composed of several convolutional blocks with increasing channel width and spatial downsampling. These blocks progressively enlarge the receptive field, allowing the network to capture both local geometric structure and broader spatial context in BEV space.

On top of the BEV backbone, we adopt a SECOND-style feature pyramid neck (SECONDFPN) [**?**], which aggregates multi-scale BEV features via top–down deconvolutional upsampling and feature concatenation. This module produces a high-resolution BEV feature map with enriched multi-scale spatial information. The SECONDFPN operates entirely in BEV space and is shared across all evaluated variants, including the LiDAR-only PointPillars baseline.

The detection head follows a single-stage, SSD-style design operating directly on the BEV feature map [**?**]. Each spatial location is associated with a set of predefined 3D anchors parameterized by class-specific sizes and canonical orientations. For each anchor, the head predicts two outputs via parallel convolutional branches:

- **Classification scores** over object classes and background,
- **3D bounding box regression** offsets

$(\Delta x, \Delta y, \Delta z, \Delta w, \Delta l, \Delta h, \Delta\theta)$ defined relative to the anchor center, dimensions, and heading,

During inference, the regression offsets are decoded to obtain absolute 3D bounding boxes in the LiDAR coordinate frame. Non-maximum suppression (NMS) is then applied in BEV space on a per-class basis to remove redundant predictions, yielding the final set of 3D detection.

3.1.7 TRAINING SETTINGS

We train FusionPointPillars (SSF, GMF and APF) and the Baseline PointPillars on the KITTI 3D object detection benchmark following widely adopted practices in PointPillars and SECOND. All models are optimized end-to-end using the AdamW optimizer with an initial learning rate of 10^{-3}, $(\beta_1, \beta_2) = (0.9, 0.999)$, and a weight decay of 10^{-2}. The learning rate follows a cosine annealing schedule with linear warm-up over the first 500 iterations and a minimum learning rate of 10^{-5}. Training is performed for 80 epochs with a batch size of 2 on a single GPU. We evaluate the model every five epochs and report the best-performing checkpoint based on the KITTI validation set, consistent with standard practice in 3D detection.

1) Data Augmentation

We adopt the strong data augmentation pipeline used by PointPillars and SECOND, which has proven essential for the KITTI dataset. Following [**?**], we construct a ground-truth database by storing all annotated 3D bounding boxes and the points they contain. During training, we randomly sample 15 car instances, 0 pedestrian instances, and 8 cyclist instances from this database and insert them into the current frame to increase dataset diversity. Each ground-truth box is further augmented individually by a small random rotation sampled from $[-\pi/20, \pi/20]$ and a random translation drawn from $\mathcal{N}(0, 0.25^2)$ along each axis.

We additionally apply global augmentations jointly to the entire point cloud and all bounding boxes, including random horizontal flipping along the x-axis, global rotation in the BEV plane, global scaling sampled from $[0.95, 1.05]$, and global translation drawn from $\mathcal{N}(0, 0.2^2)$ in (x, y, z). Points inside each pillar are randomly shuffled to prevent the network from exploiting point ordering. When camera images are used, identical flip and geometric transformations are applied to the RGB image to maintain geometric consistency across modalities.

2) Pillarization

We follow the standard KITTI configuration for LiDAR discretization. The 3D space is partitioned into vertical pillars of size

$$(v_x, v_y, v_z) = (0.16 \text{ m}, 0.16 \text{ m}, 4.0 \text{ m}), \tag{18}$$

within the point cloud range

$$x \in [0, 69.12], \quad y \in [-39.68, 39.68], \quad z \in [-3, 1]. \tag{19}$$

Each non-empty pillar stores up to 100 points and is encoded by a Pillar Feature Network (PFN) to produce a 64-dimensional embedding, which is subsequently fused with image features as described in Sec. **??**.

3) Image Preprocessing

Images are normalized to $[0, 1]$ and processed by a ResNet-50 backbone with an FPN, producing multi-scale feature maps $\{P_2, P_3, P_4\}$. Projected pillar centers are aligned with these feature maps via the KITTI LiDAR–camera calibration matrix, and a 3×3 ROI pooling operation is performed on each level to extract localized semantic features for multimodal fusion.

4) Loss Functions

FusionPointPillars adopts the loss formulation of PointPillars, including focal loss for classification, Smooth L1 loss for bounding box regression, and a direction classification loss to resolve heading ambiguity. Detection is performed using class-specific 3D anchors. Regression offsets are decoded into absolute 3D bounding boxes in LiDAR coordinates, and non-maximum suppression (NMS) is applied in BEV space to produce the final predictions.

3.1.8 EVALUATION

We evaluate FusionPointPillars on the KITTI 3D object detection benchmark [**?**], following the standard training and validation splits. The dataset contains three object categories: *Car*, *Pedestrian*, and *Cyclist*, with annotations provided in both camera and LiDAR coordinate frames. Performance is reported using the official KITTI evaluation metrics for bird's-eye view (BEV) and 3D object detection under three difficulty levels: easy, moderate, and hard.

For Cars, an IoU threshold of 0.7 is used, while for Pedestrians and Cyclists an IoU threshold of 0.5 is adopted, consistent with prior work. We report average precision using the AP40 protocol, which samples 40 recall positions and has become the standard for recent KITTI evaluations.

3.2 FUSION OF POINTPILLARS WITH BEVFORMER

The second architectural paradigm introduces a transformer-centric multimodal framework that synergizes PointPillars-based LiDAR feature extraction with the BEVFormer multi-view camera encoding scheme [**?**]. Rather than performing early feature-level fusion at the pillar representation stage, this design strategically injects LiDAR information into the Bird's-Eye-View (BEV) representation at two complementary stages within the transformer pipeline. Furthermore, a dedicated velocity head is incorporated to isolate the temporal motion signal inherent in the camera stream from the static geometric information provided by LiDAR. To investigate the optimal depth of multi-sensor integration, we define two primary architectural configurations: a Decoder-only fusion model and a dual-stage Encoder-Decoder fusion model.

3.2.1 CORE ARCHITECTURAL VARIANTS

The distinction between these two models lies in the frequency and method of interaction between the LiDAR and camera modalities throughout the transformer pipeline.

- *Decoder-only Fusion Model (V3):* In this variant, the BEV encoding process remains exclusively camera-based, utilizing standard spatial cross-attention (SCA) and temporal self-attention (TSA). LiDAR information is introduced only after the encoder has produced its output. This injection happens at the decoder stage, where the LiDAR BEV map is merged with the camera-dominant BEV snapshot to provide final geometric context for the object queries. The complete data flow of this variant is illustrated in Fig. **??**.
- *Encoder-Decoder Fusion Model (Enc_Dec):* Identified as the *Enc_Dec* variant, this model employs a dual-injection strategy. Beyond the decoder-side fusion, it also incorporates LiDAR into every layer of the transformer encoder. Each encoder layer is modified with a parallel LiDAR spatial cross-attention branch, allowing for iterative, attention-guided blending of geometric and semantic data throughout the entire encoding process. An overview of this dual-stage fusion pipeline is shown in Fig. **??**.

By comparing these two models, we can distinguish between the benefits of late-stage geometric correction (Decoder-only) and the advantages of continuous, multi-layer modality interaction (Encoder-Decoder). The following sections detail the specific implementation of the sensor branches and these fusion mechanisms.

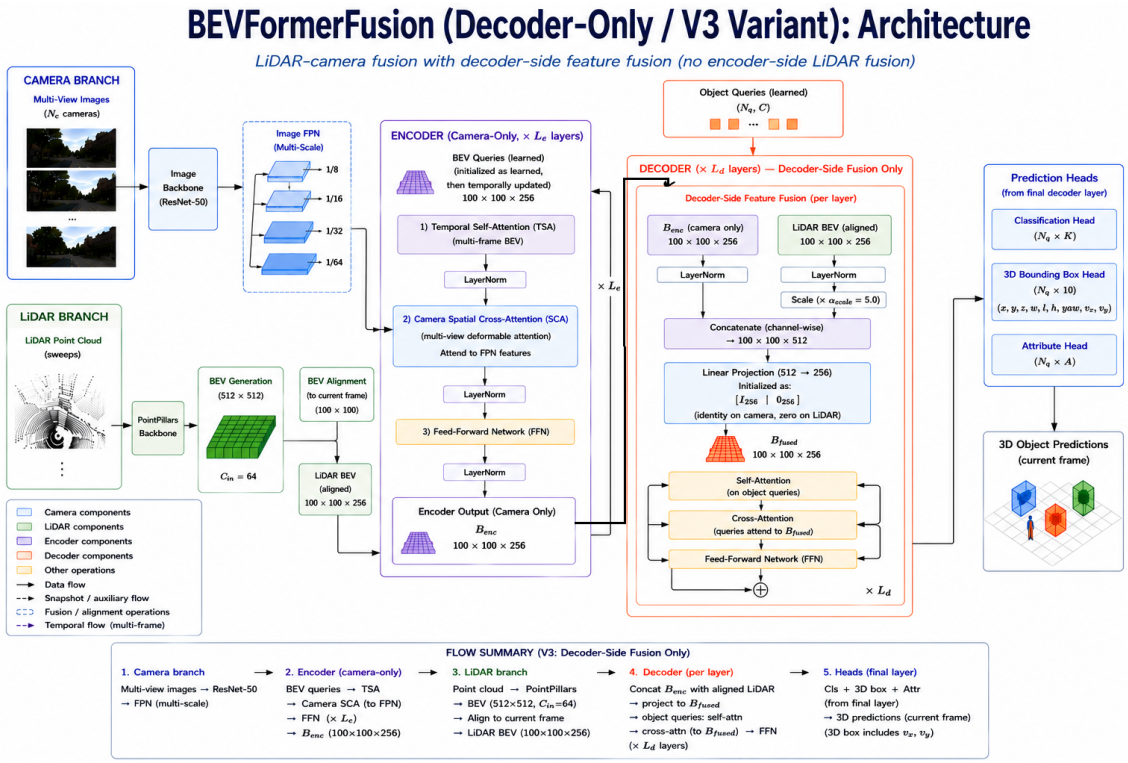


Fig. 2. Overview of the BEVFormerFusion (Decoder-only / V3 variant) architecture. The encoder operates exclusively on camera features (ResNet-50 + FPN) through Temporal Self-Attention and Camera Spatial Cross-Attention, producing a camera-only BEV output B_{enc}. LiDAR BEV features (PointPillars) are injected only at the decoder stage via channel-wise concatenation with B_{enc} to form B_{fused}, which serves as the cross-attention context for the object queries.

3.2.2 CAMERA BRANCH

Six surround-view images are processed by a shared ResNet-50 backbone [**?**], pre-trained on ImageNet. Four feature maps at different spatial resolutions are extracted and further refined by a Feature Pyramid Network (FPN) [**?**] to produce four multi-scale feature volumes, each with 256 channels. During training, GridMask augmentation [**?**] is applied to enhance robustness against occlusions.

The BEV representation is structured as a set of learnable queries defined over a 100×100 spatial grid, covering a point cloud range of $[-51.2, -51.2, -5.0, 51.2, 51.2, 3.0]$ m. Each BEV query interacts with multi-view image features through a Spatial-Cross-Attention (SCA) mechanism. Specifically, BEV reference points–sampled at four pillar heights per grid cell– are projected into the camera image plane using calibrated LiDAR-to-image transformation matrices. Only queries projecting within the visible view frustum of a given camera contribute to its attention computation. Furthermore, temporal context is incorporated through a Temporal Self-Attention (TSA) module. This module aligns the

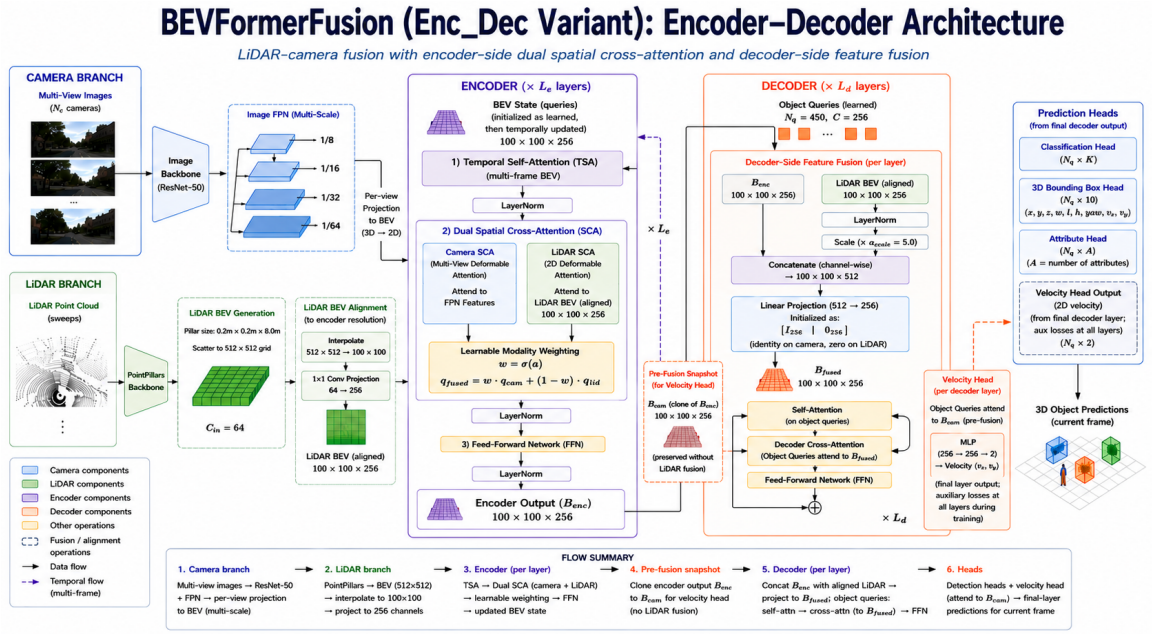


Fig. 3. Overview of the BEVFormerFusion (Enc_Dec variant) architecture. Camera features (ResNet-50 + FPN) and LiDAR BEV features (PointPillars) are fused at two stages: within each encoder layer via Dual Spatial Cross-Attention with learnable per-layer weighting $w = \sigma(\alpha)$, and again before the decoder via channel-wise concatenation to form B_{fused}. A pre-fusion snapshot B_{cam} is preserved as the cross-attention context for the velocity head.

current BEV query set with the stored BEV embedding from the previous timestep after ego-motion compensation, thereby accumulating inter-frame motion cues across a queue of four consecutive frames.

3.2.3 LIDAR BRANCH

The LiDAR branch leverages the PointPillars architecture [**?**] to convert the input point clouds into a structured, dense BEV feature tensor. The input (raw LiDAR) points are first voxelized into pillars of size $0.2\,\text{m} \times 0.2\,\text{m} \times 8.0\,\text{m}$ (a single height bin), with at most 20 points per pillar and a training limit of 30,000 pillars. A PillarFeatureNet with one fully-connected layer encodes per-pillar descriptors to 64 dimensions using PointNet-style channel-wise max-pooling. A PointPillarScatter module then maps the encoded descriptors onto a 512×512 2D spatial grid, yielding a dense LiDAR BEV tensor of shape $(B, 64, 512, 512)$. This tensor serves as the shared input to both fusion stages below. PointPillars was selected for its computational efficiency and for producing an axis-aligned BEV tensor whose spatial coordinate system is directly compatible with the BEVFormer BEV grid.

3.2.4 ENCODER-SIDE FUSION

Each of the four BEV encoder layers is implemented as an `MM_BEVFormerLayer` that extends the standard BEVFormer encoder layer with a parallel LiDAR spatial cross-attention branch (see the encoder block in Fig. **??**). The computational sequence within each layer is: Temporal Self-Attention $\rightarrow$ LayerNorm $\rightarrow$ Dual Spatial Cross-Attention $\rightarrow$ LayerNorm $\rightarrow$ Feed-Forward Network $\rightarrow$ LayerNorm.

Within the Dual SCA stage, BEV queries simultaneously aggregate information from two distinct modalities: (1) multi-view camera features through the standard 3D multi-scale deformable attention mechanism [**?**], and (2) projected LiDAR BEV tokens via a flat 2D deformable attention module (`CustomMS-DeformableAttention`, configured with 8 heads, 4 sampling points, 1 spatial level). To ensure spatial and dimensional alignment, the 64-channel LiDAR BEV map is spatially interpolated to 100×100 and projected to a 256-channel embedding space using a 1×1 convolutional layer prior to the encoder.

The camera SCA and LiDAR SCA outputs are merged through a per-layer learnable scalar weight $w = \sigma(\alpha)$:

$$\mathbf{q}_{\text{fused}} = w \cdot \mathbf{q}_{\text{cam}} + (1 - w) \cdot \mathbf{q}_{\text{lid}}, \tag{20}$$

where α is a trainable parameter initialized to zero, corresponding to an initial equal contribution ($w = 0.5$) from both modalities. Each of the four encoder layers maintains its own independent weight, enabling the network to dynamically recalibrate the camera–LiDAR balance across varying depths of feature abstraction.

3.2.5 DECODER-SIDE FUSION

After the BEV encoder produces its output embedding $\mathbf{B}_{\text{enc}} \in \mathbb{R}^{B \times 10000 \times 256}$, a second LiDAR injection stage is employed to enrich the feature representation prior to the transformer decoder (decoder block in Figs. **??** and **??**). The encoder output is first cloned to preserve a camera-dominant BEV snapshot $\mathbf{B}_{\text{cam}}$ (discussed in §**??**). Concurrently, the LiDAR BEV map is interpolated to 100×100, projected to 256 channels, normalized via LayerNorm. To ensure magnitude compatibility with the camera features, these tokens are scaled by a constant factor $\alpha_{\text{scale}} = 5.0$.

The normalized LiDAR tokens are subsequently concatenated channel-wise with the encoder output, yielding a 512-channel tensor. A linear projection layer ($\mathbb{R}^{512} \rightarrow \mathbb{R}^{256}$) with subsequent LayerNorm produces the fused BEV $\mathbf{B}_{\text{fused}}$ that serves as the cross-attention context for the 450 object queries of the decoder.

The fusion linear layer is initialized with identity weights on the camera-side columns and zeros for the LiDAR-side columns. This initialization strategy ensures that the model begins training as a camera-only baseline and gradually learns to incorporate geometric information from the LiDAR stream as training progresses.

3.2.6 VELOCITY HEAD WITH A PRE-DECODER-FUSION BEV SNAPSHOT

Velocity estimation relies heavily on the inter-frame displacement signals encoded by the Temporal Self-Attention(TSA) module. Given that the LiDAR point cloud captures a single timestep, injecting LiDAR features prior to velocity prediction tends to dilute these temporal cues with static geometric information. To mitigate this, the cloned encoder output $\mathbf{B}_{\text{cam}}$—taken before decoder-side fusion—is used as the key-value context for a dedicated velocity cross-attention module (Pre-Fusion Snapshot and Velocity Head blocks in Fig. **??**).

Within each of the six decoder layers, a multi-head attention block (8 heads, 256 dimensions) enables the current decoder queries to attend to the full 10,000-token $\mathbf{B}_{\text{cam}}$. This is followed by a two-layer MLP ($256 \to 256 \to 2$) that generates per-object velocity predictions (v_x, v_y). The velocity head is trained with Smooth L1 (Huber) loss (with a loss weight of 0.25) applied exclusively to matched ground-truth queries. Furthermore, the velocity gradient is isolated by zeroing the velocity channels in the bounding-box regression loss.

3.2.7 TRAINING CONFIGURATION

3D object Detection is supervised using a set-prediction paradigm [**?**], where 450 object queries are matched to ground-truth boxes via Hungarian assignment across ten nuScenes object classes. To handle the periodicity of orientations, the yaw angle is decomposed into a 24-bin classification task combined with a $(\sin, \cos)$ residual regression, effectively eliminating discontinuities at $\pm\pi$.

Here, the matching between predictions and ground-truth boxes is obtained by the Hungarian algorithm under the set-prediction paradigm. The yaw angle is decomposed into a discrete bin classification term and a continuous residual regression term. Auxiliary supervision is applied at all six decoder layers using the same loss formulation.

$$\mathcal{L}_{\text{total}} = \lambda_{\text{cls}}\mathcal{L}_{\text{focal}} + \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{yaw_cls}}\mathcal{L}_{\text{yaw_cls}} + \lambda_{\text{yaw_reg}}\mathcal{L}_{\text{yaw_reg}} + \lambda_{\text{vel}}\mathcal{L}_{\text{vel}}. \quad (21)$$

$$\mathcal{L}_{\text{box}} = \|b - \hat{b}\|_1, \tag{22}$$

$$\mathcal{L}_{\text{yaw_cls}} = \mathcal{L}_{\text{CE}}(\theta^{\text{bin}}, \hat{\theta}^{\text{bin}}), \tag{23}$$

$$\mathcal{L}_{\text{yaw_reg}} = \mathcal{L}_{\text{SmoothL1}}(\Delta\theta, \Delta\hat{\theta}), \tag{24}$$

$$\mathcal{L}_{\text{vel}} = \mathcal{L}_{\text{SmoothL1}}(v, \hat{v}). \tag{25}$$

The total loss is a weighted combination of Focal Loss (classification, $\lambda = 2.0$), L1 loss (bounding-box regression, $\lambda = 0.25$), Cross-Entropy (yaw bin, $\lambda = 0.2$), Smooth L1 (yaw residual, $\lambda = 0.2$), and Smooth L1 (velocity, $\lambda = 0.25$), with auxiliary supervision applied at all six decoder layers.

Training is conducted on the nuScenes dataset [**?**] for 100,000 iterations using the AdamW optimizer. We adopt an initial learning rate lr $= 2 \times 10^{-4}$ with a weight decay of 0.01, and a gradient clipping at max-norm 1.0. A $0.1 \times multiplier$ is applied to the image backbone's learning rate to maintain pre-trained feature stability. The learning rate schedule follows a cosine annealing policy, preceded by a linear warmup over the first 10,000 iterations, decaying to a minimum ratio of 10^{-3}. All experiments are conducted in full FP32 precision with a batch size of 1 per GPU.

4 EXPERIMENTAL RESULTS

4.1 RESULTS OF FUSIONPOINTPILLARS

We evaluated the following four configurations and compared the results with MVXNet, and Multimodal PointPillars (M-PP).

- **Baseline PointPillars (BaselinePP)**
- **Single-Scale Fusion (SSF)**
- **Global Multi-Scale Fusion (GMF)**
- **Adaptive Pillar Fusion (APF)**

4.1.1 3D OBJECT DETECTION RESULTS

Table **??** reports the AP40 results for 3D object detection on the KITTI validation set. All three FusionPointPillars variants achieve performance comparable to the LiDAR-only baseline for Car detection, although the baseline slightly outperforms the fusion models at the Easy level. This behavior is expected, as large objects such as Cars are already well represented by dense LiDAR geometry, and the addition of image features introduces minor feature interference without providing significant additional benefit. FusionPointPillars (SSF) shows slightly better results than two other FusionPointPillars (GMF and APF) for Cars. In contrast, for the Pedestrian category, FusionPointPillars (GMF) consistently outperforms SSF and APF across all difficulty levels. Different results are observed for Cyclists, where GMF and APF yield almost the same best overall 3D performance compared to both SSF and LiDAR-only baseline. These results demonstrate that multi-scale fusion, either global or adaptive, are especially beneficial for small or sparsely observed objects, for which LiDAR point density alone is often insufficient for reliable 3D localization.

Overall, the 3D detection results indicate that the simple SSF model is effective for large objects such as Cars, while the learnable GMF model provides clear advantages for Pedestrians. GMF and APF are almost equally effective for Cyclists. These improvements by learnable fusion allow the network to adaptively emphasize different semantic scales depending on object category.

4.1.2 BEV OBJECT DETECTION RESULTS

Table **??** summarizes the BEV detection performance on the KITTI validation set. Multimodal fusion significantly improves small object detection in BEV space. In particular, FusionPointPillars (GMF) achieves the highest Pedestrian BEV AP among the evaluated pillar-based methods, indicating that multi-scale semantic context significantly improves

TABLE 1: KITTI validation AP40 for 3D object detection.

Method	Car			Ped			Cyc		
	E	M	H	E	M	H	E	M	H
MVX-Net (VF) [4]	82.3	72.2	66.8	–	–	–	–	–	–
MVX-Net (PF) [4]	85.5	73.3	67.4	–	–	–	–	–	–
PP [18]	79.05	74.99	68.30	–	–	–	–	–	–
BaselinePP	**85.78**	72.59	69.93	52.05	45.80	41.26	73.65	52.95	49.68
SSF	84.05	**74.97**	**70.95**	49.46	43.40	39.14	75.52	58.97	55.47
GMF	83.16	74.13	70.31	**55.49**	**48.82**	**44.01**	79.27	60.07	56.14
APF	83.60	74.71	70.68	52.04	44.93	40.08	**79.90**	**60.33**	**56.88**

TABLE 2: KITTI validation AP40 for BEV object detection.

Method	Car			Ped			Cyc		
	E	M	H	E	M	H	E	M	H
MVX-Net (VF) [4]	88.6	84.6	78.6	–	–	–	–	–	–
MVX-Net (PF) [4]	89.5	84.9	79.0	–	–	–	–	–	–
M-PP [5]	89.11	**87.03**	79.51	**69.29**	**62.78**	**60.28**	80.03	60.93	57.69
PP [18]	88.35	86.10	79.83	58.66	50.23	47.19	79.14	62.25	56.00
BaselinePP	**93.89**	85.48	**82.96**	58.27	52.33	48.64	79.51	59.45	56.02
SSF	90.53	85.20	82.61	55.75	50.12	45.71	80.56	64.46	60.78
GMF	87.91	84.02	80.06	61.32	54.95	50.56	81.36	64.04	60.04
APF	90.32	84.99	82.58	59.60	52.33	47.43	**83.34**	**64.94**	**61.86**

ground-plane localization and object extent estimation. For the Cyclist category, all three models (SSF, GMF and APF) achieve the stronger overall BEV performance than LiDAR-only baselines across all difficulty settings. Among three models, APF yields the best overall BEV performance. We attribute this behavior to the proposed pillar-wise fusion strategy combined with learnable multi-scale image aggregation, which is well suited for elongated and thin objects such as cyclists.

For Cars, the Baseline PointPillars model achieves the highest BEV AP overall, suggesting that BEV-scale aggregation of LiDAR features alone is already highly effective for large objects. SSF and APF consistently outperform GMF in Car BEV AP across all levels of difficulty. These findings are consistent with our observations in 3D detection.

4.1.3 COMPARISON WITH EXISTING LIDAR-CAMERA FUSION METHODS

To assess the effectiveness of the proposed fusion strategies, we compare our results with representative LiDAR–camera fusion methods, including MVX-Net [**?**] and Multimodal PointPillars (M-PP) [**?**].

1) Comparison with MVX-Net

MVX-Net introduces early voxel-level fusion via two variants: PointFusion (PF) and VoxelFusion (VF). On the KITTI benchmark, MVX-Net reports 3D AP (Car, moderate) of 72.2 % (VoxelFusion) and 73.3 % (PointFusion). In comparison, our SSF, GMF, and APF each achieve more than 74 % (Car, moderate, 3D AP40), while maintaining a pillar-based architecture without 3D sparse convolutions. This comparison highlights two important differences:

- MVX-Net relies on voxel-level fusion with 3D convolutions, increasing computational complexity.
- APF performs fusion at the pillar level and operates entirely in BEV space, preserving the computational advantages of PointPillars.

Despite its lightweight design, FusionPointPillars achieves competitive or superior Car detection performance compared to MVX-Net, while demonstrating stronger improvements for smaller object classes such as Cyclists.

2) Comparison with Multimodal PointPillars (M-PP)

Compared to M-PP, which provided only AP40 for BEV detection and not for 3D detection, FusionPointPillars delivers overall competitive BEV performance, although its Pedestrian BEV AP remains lower. This difference reflects the distinct design philosophies of the two approaches: M-PP emphasizes aggressive point-wise fusion to maximize pedestrian recall, whereas FusionPointPillars prioritizes architectural simplicity, computational efficiency, and end-to-end pillar-wise fusion. Notably, for the Cyclist category, all three FusionPointPillars variants (SSF, GMF and APF) achieve higher BEV performance than M-PP, particularly at the moderate and hard difficulty level.

4.1.4 QUALITATIVE RESULTS

Figure **??** presents a qualitative detection example produced by FusionPointPillars (GMF). Predicted 3D bounding boxes are projected from the LiDAR coordinate frame onto the camera image for visualization. The model successfully detects vehicles across a wide range of distances and viewpoints, including partially occluded and side-parked cars.

It is important to note that projected 3D bounding boxes may appear slightly misaligned with object silhouettes in the image. This effect is expected, as predictions are made in LiDAR space and KITTI annotations represent physical object extents rather than

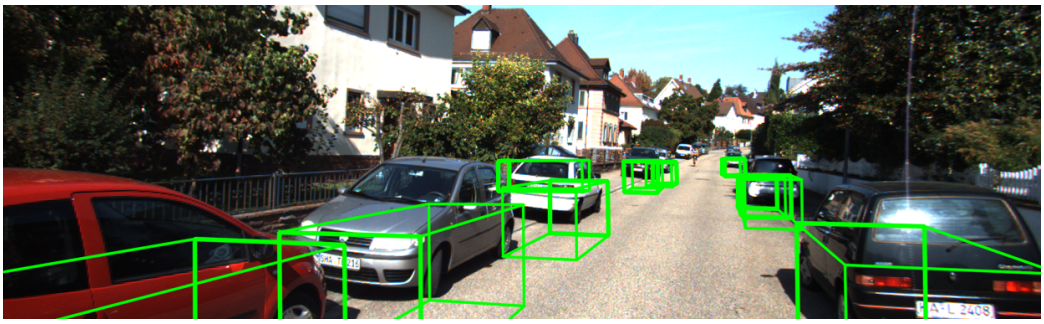


Fig. 4. Example detections produced by FusionPointPillars (GMF) on the KITTI dataset. Green boxes indicate predicted 3D bounding boxes projected onto the camera image. Apparent offsets between projected boxes and object silhouettes are expected due to LiDAR–camera parallax and do not affect BEV or 3D detection accuracy.

image-tight regions. Additionally, LiDAR–camera parallax introduces systematic offsets, particularly for nearby objects.

Figure **??** shows a BEV visualization of predicted and ground-truth bounding boxes obtained from the same sample. Predicted boxes without matching ground-truth annotations correspond to objects labeled as *DontCare* in the KITTI dataset, which are intentionally excluded from quantitative evaluation. Such predictions are penalized as false positives during evaluation and therefore do not inflate AP metrics. These examples illustrate the model's ability to detect visually and geometrically consistent objects beyond the subset of annotated targets.

4.1.5 INFERENCE SPEED ANALYSIS

To assess the computational overhead introduced by multimodal fusion, we measure the inference speed of FusionPointPillars and compare it with the LiDAR-only PointPillars baseline. All timing experiments are conducted on a single NVIDIA GeForce RTX 3060 GPU with 12 GB memory, paired with an Intel® Core™ i5-10400 CPU and 32 GB system RAM. The models are evaluated using batch size 1 after warm-up iterations, and the reported values are averaged over multiple runs to ensure stable timing.

Table **??** summarizes the inference latency and throughput. The LiDAR-only PointPillars baseline achieves an average latency of 37.50 ms per frame (26.67 FPS), reflecting the efficiency of BEV-based pillar processing without image fusion. Introducing image-based fusion increases computational cost due to the additional image backbone and FPN feature extraction. SSF incurs an average latency of 84.07 ms, while GMF and APF achieve similar latencies of 82.95 ms and 83.36 ms, respectively.

The comparable runtime of all three FusionPointPillars variants indicates that the dominant computational overhead arises from the image backbone and feature extraction

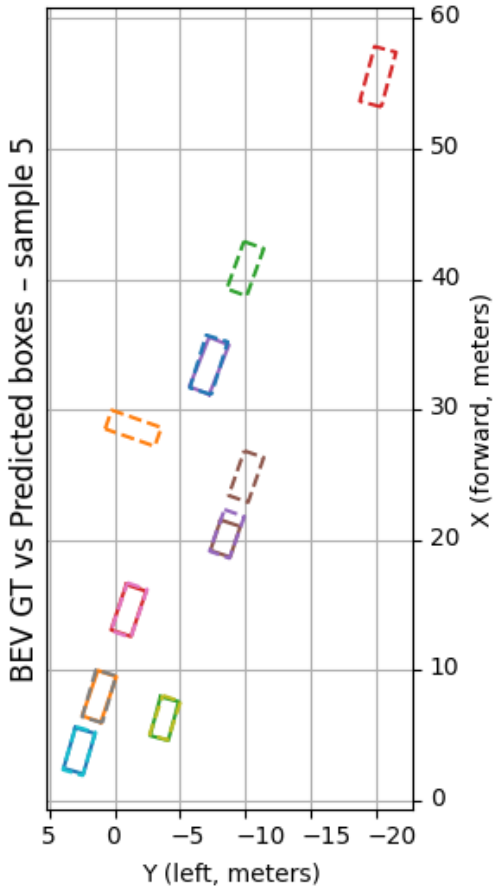


Fig. 5. Bird's-eye-view (BEV) visualization of ground-truth and predicted 3D bounding boxes of the example shown in Figure 4. Ground-truth boxes are shown with solid outlines, while predicted boxes from FusionPointPillars (GMF) are shown with dashed outlines. Four predicted boxes without corresponding GT boxes are objects labeled as *DontCare*. The figure illustrates spatial alignment between predictions and annotations on the ground plane, highlighting the model's ability to localize objects accurately in BEV space.

pipeline, whereas the additional multi-scale aggregation and adaptive weighting mechanisms introduce only negligible extra cost.

During inference of the ResNet-50 GMF model, the learned fusion weights converged to approximately

$$\alpha = [0.38,\ 0.32,\ 0.30]$$

for (P_2, P_3, P_4), respectively, indicating that the network leverages information from all feature scales while slightly favoring high-resolution image features.

To further investigate the relationship between backbone complexity, inference speed, and detection performance, we additionally evaluated GMF using a lighter ResNet-18

TABLE 3: Inference speed comparison on the KITTI validation set.

Model	**Backbone**	**Latency (ms)**	**FPS**
BaselinePP	LiDAR only	37.50	26.67
SSF	ResNet-50	84.07	11.89
GMF	ResNet-50	82.95	12.05
APF	ResNet-50	83.36	11.98
GMF	ResNet-18	70.92	14.10

TABLE 4: ResNet-18 GMF 3D AP40 comparison at 15 and 25 epochs.

Epoch	**Car**			**Pedestrian**			**Cyclist**		
	Easy	Mod.	Hard	Easy	Mod.	Hard	Easy	Mod.	Hard
15	79.18	69.54	66.10	48.19	41.45	37.64	74.34	55.70	52.44
25	71.19	64.51	62.44	39.25	33.62	29.57	69.74	53.47	50.16

image backbone. As shown in Table 3, the ResNet-18 GMF model achieves 14.1 FPS (70.92 ms latency), representing a substantial improvement over the ResNet-50-based fusion models. This result further confirms that the image backbone is the primary contributor to multimodal inference cost.

Table **??** compares the 3D AP40 results of the ResNet-18 GMF model at 15 and 25 epochs. The model achieves its best performance at 15 epochs across all object categories, while performance degrades noticeably at 25 epochs, particularly for Pedestrian and Car detection. Furthermore, even at its best checkpoint, the ResNet-18 model remains inferior to the corresponding ResNet-50-based GMF model reported in Table **??**. These observations suggest that the lighter backbone provides weaker semantic feature representations and is more sensitive to overfitting during longer training.

The learned fusion weights of the ResNet-18 GMF model were approximately

$$\alpha = [0.505,\ 0.283,\ 0.212]$$

for (P_2, P_3, P_4), respectively, indicating a stronger dependence on the highest-resolution feature level P_2. In contrast, the ResNet-50 GMF model produced a more balanced distribution across pyramid levels, suggesting more effective utilization of multi-scale semantic information.

For reference, Multimodal PointPillars (M-PP) reports an inference speed of 28.49 FPS on a significantly more powerful Tesla V100-SXM2 GPU with 32 GB memory and a Xeon Gold 6150 CPU. Although direct numerical comparison is not fully meaningful due to hardware differences, this comparison provides useful context for the computational

cost of multimodal fusion. MVX-Net does not report inference speed; however, its reliance on voxel-based 3D convolutions generally implies higher computational complexity than pillar-based approaches.

Overall, FusionPointPillars increases inference latency compared with the LiDAR-only baseline, but maintains practical real-time performance. The ResNet-18 experiment further demonstrates that lighter image backbones can substantially improve throughput, although this comes at the cost of reduced accuracy and training stability.

4.2 RESULTS OF FUSION OF POINTPILLARS WITH BEVFORMER

4.2.1 BENCHMARK PROTOCOL

All BEVFormerFusion experiments are evaluated on the nuScenes validation split using the official nuScenes detection metrics [**?**]. All experiments are conducted on a system equipped with a single NVIDIA GeForce RTX 3060 GPU with 12 GB memory, an Intel® Core™ i5-10400 CPU (2.90 GHz), and 32 GB system RAM. Primary metrics are mean Average Precision (mAP) and the nuScenes Detection Score (NDS), a composite combining mAP with five per-attribute error terms: mean Average Translation Error (mATE), Scale Error (mASE), Orientation Error (mAOE), Velocity Error (mAVE), and Attribute Error (mAAE). Lower values are better for all five error terms. Ten object classes are evaluated: car, truck, bus, trailer, construction vehicle, pedestrian, motorcycle, bicycle, traffic cone, and barrier. It is important to note that the dataset, sensor configuration, and metric family used in this section differ entirely from the KITTI/AP40 evaluation used for the FusionPointPillars track; no direct cross-track numerical comparison is made. Model checkpoints are saved every 10,000 iterations; the best checkpoint per variant is selected by NDS on the validation set.

Training, evaluation, and latency benchmarking were conducted on a local machine equipped with one NVIDIA GeForce RTX 3060 GPU (12 GB), an Intel® Core™ i5-10400 CPU (2.90 GHz), and 32 GB RAM. Although Table 5 reports representative single-run results for each BEVFormerFusion variant, additional runs were conducted under the same experimental setup to assess result stability. The repeated runs produced results within 2% of the values reported in Table 5, indicating that the observed trends were consistent.

4.2.2 EXPERIMENTAL VARIANTS

Three variants are reported. **Baseline** is a camera-only BEVFormer model using the same backbone, BEV grid (100×100), and training schedule as the fusion variants, with LiDAR input disabled. **Enc_Dec** is the full encoder-decoder fusion model described in §**??**, in which LiDAR features are injected both within the BEV encoder and again before

TABLE 5: BEVFormerFusion variants on the nuScenes validation split. Best NDS checkpoint shown. ↓ denotes lower is better.

Variant	Ckpt	mAP↑	NDS↑	mATE↓	mASE↓	mAOE↓	mAVE↓	mAAE↓
Baseline (cam-only)	100k	0.201	0.219	0.949	0.716	0.815	1.042	0.333
Enc_Dec	100k	**0.251**	**0.255**	**0.870**	0.717	0.824	1.138	**0.296**
V3	90k	0.202	0.214	0.872	0.720	0.857	1.139	0.424

the transformer decoder. **Decoder (V3)** is an improved decoder-side fusion model that injects LiDAR features only after BEV encoding and before decoder cross-attention. This variant was developed to test whether a lighter-weight fusion strategy could retain some of the geometric benefit of LiDAR while remaining closer to the camera-only baseline in runtime.

4.2.3 QUANTITATIVE RESULTS

Table **??** reports headline metrics for all variants. Enc_Dec achieves the highest NDS (0.2546) and mAP (0.2507), representing gains of $+0.035$ NDS and $+0.050$ mAP over the camera-only Baseline at equivalent training length. Translation error (mATE) improves substantially from 0.949 (Baseline) to 0.870 (Enc_Dec), consistent with LiDAR providing precise range measurements. Attribute error (mAAE) also improves. Velocity error (mAVE) is marginally higher for Enc_Dec (1.138) than for the Baseline (1.042), a result that is discussed further in §**??**. Decoder (V3) does not improve over the Baseline in NDS under the evaluated conditions, although it shows lower mATE than the Baseline, suggesting that decoder-side LiDAR fusion alone can improve localization quality. However, its overall detection quality remains below Enc_Dec. Taken together, these results indicate that decoder-side fusion is feasible and computationally attractive, but the largest overall gains are obtained when encoder-side and decoder-side fusion are combined.

To complement Table 5, Figure **??** visualizes the metric values for Baseline, Enc_Dec, and Decoder (V3) across the seven nuScenes evaluation metrics. The grouped bars make the main trend easier to see: Enc_Dec achieves the best overall detection metrics (mAP and NDS) and the lowest translation error (mATE), while Decoder (V3) remains closer to the Baseline in overall profile. Figure **??** provides a compact radar-style summary of the same results. Because five of the seven axes are error metrics for which lower values are better, Figure 7 should be interpreted together with Table 5 rather than as an area-based ranking.

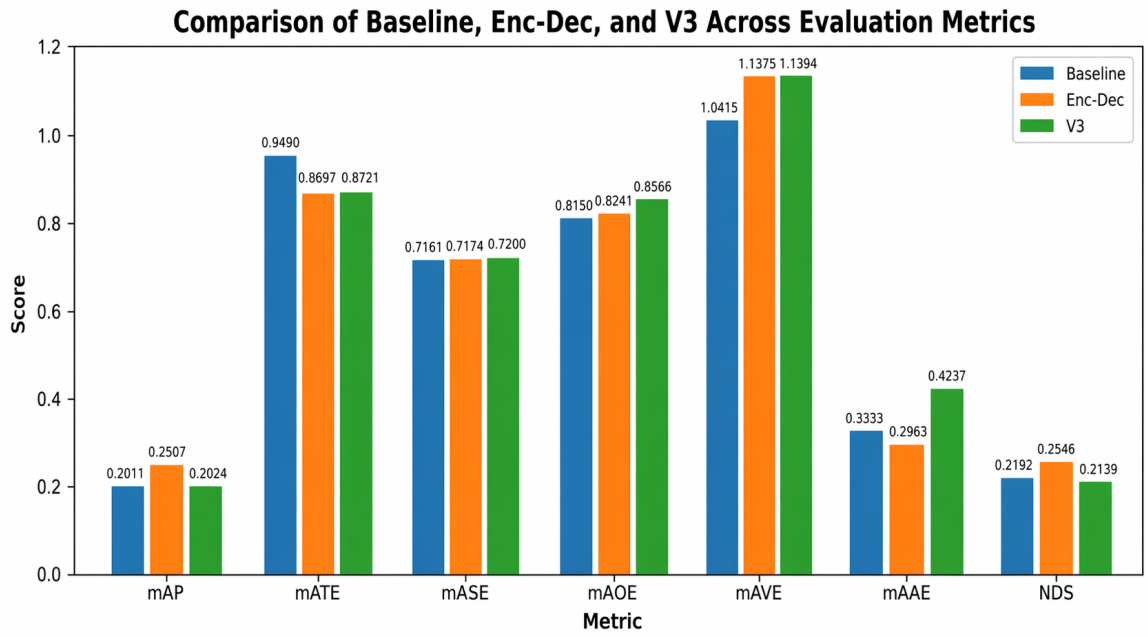


Fig. 6. Comparison of Baseline, Enc_Dec, and Decoder (V3) across nuScenes evaluation metrics. Higher is better for mAP and NDS; lower is better for mATE, mASE, mAOE, mAVE, and mAAE. Enc_Dec achieves the strongest overall detection performance, while Decoder (V3) remains closer to the camera-only baseline.

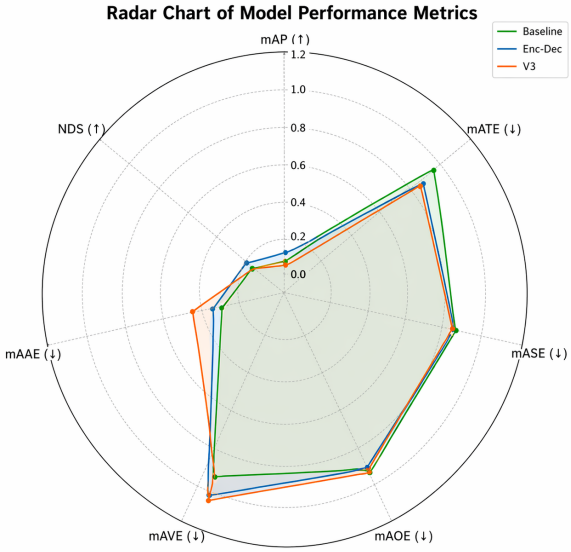


Fig. 7. Radar-chart summary of Baseline, Enc_Dec, and Decoder (V3) on the nuScenes validation metrics. The figure is intended as a qualitative visual summary only. Because mATE, mASE, mAOE, mAVE, and mAAE are error metrics, lower values indicate better performance.

4.2.4 PER-CLASS ANALYSIS

Table **??** reports per-class AP for the Baseline and Enc_Dec at their respective best NDS checkpoints (both at iteration 100k). Enc_Dec shows numerical AP gains in nine of the

TABLE 6: Per-class AP on nuScenes val split. Baseline (100k) vs Enc_Dec (100k).

Class	Baseline	Enc_Dec	Δ
car	0.359	0.424	+0.065
truck	0.090	0.152	+0.062
bus	0.161	0.314	+0.153
trailer	0.019	0.046	+0.027
construction vehicle	0.068	0.068	0.000
pedestrian	0.289	0.339	+0.050
motorcycle	0.203	0.240	+0.037
bicycle	0.196	0.198	+0.002
traffic cone	0.367	0.385	+0.018
barrier	0.261	0.343	+0.082

ten classes, while construction vehicle remains unchanged. The largest absolute gains are recorded for bus (+0.153), barrier (+0.082), car (+0.065), and truck (+0.062). These classes are large objects with strong, spatially compact LiDAR returns. Construction vehicle remains unchanged and bicycle improves only marginally, which may reflect insufficient training-set coverage or the limited discriminability of LiDAR pillar features for these categories under the current voxelization.

4.2.5 ENCODER-SIDE LIDAR CONTRIBUTION DYNAMICS IN ENC_DEC

For the Enc_Dec variant, the logged encoder-side LiDAR contribution ratio was monitored throughout training. At iteration 1,000, the logged LiDAR contribution is approximately 1.2%. It rises progressively, reaching about 11% by iterations 90,000–100,000. This trajectory indicates that the model initially relies almost entirely on the camera branch and gradually learns to incorporate LiDAR geometry as optimization proceeds. Because the logged quantity is not identical to the raw sigmoid blend parameter in Eq. (**??**), we interpret this trend qualitatively rather than as an absolute modality weight.

4.2.6 LATENCY AND EFFICIENCY ANALYSIS

To evaluate the practical cost of fusion, we benchmarked inference latency for the three BEVFormerFusion variants on a local machine equipped with one NVIDIA GeForce RTX 3060 GPU (12 GB), an Intel® Core™ i5-10400 CPU (2.90 GHz), and 32 GB RAM. Each model was evaluated on 200 samples, and we report the average latency in both seconds and milliseconds, together with throughput in frames per second (FPS).

Table **??** shows that the camera-only Baseline achieves 259.32 ms latency and 3.86 FPS. The Decoder (V3) variant remains close to the Baseline at 269.31 ms and 3.71

TABLE 7: Latency and Efficiency Analysis of BEVFormer-based Models.

Variant	Samples	Average Latency (s)	Average Latency (ms)	FPS
Baseline	200	0.2593	259.32	3.86
Enc_Dec	200	0.4315	431.47	2.32
Decoder (V3)	200	0.2693	269.31	3.71

FPS, indicating that decoder-side fusion adds only modest runtime overhead. In contrast, Enc_Dec reaches the highest accuracy but also the highest latency, 431.47 ms and 2.32 FPS. This result suggests a clear trade-off: full two-stage fusion provides the strongest detection performance, while decoder-side-only fusion offers a more conservative balance between geometric enhancement and runtime cost.

5 DISCUSSION

5.1 FUSIONPOINTPILLARS

The experimental results demonstrate that FusionPointPillars effectively leverages camera information to enhance pillar-based LiDAR detection, particularly for small and sparsely observed object categories. Across both BEV and 3D evaluation metrics, the proposed fusion strategy yields consistent improvements for pedestrians and cyclists, confirming the benefit of incorporating semantic image cues into the pillar representation.

For the car category, FusionPointPillars achieves performance comparable to the LiDAR-only PointPillars baseline, with only marginal gains under the strict 3D IoU threshold of 0.7. This behavior is expected, as cars are large, rigid objects that typically generate dense LiDAR returns, allowing accurate localization to be achieved primarily from geometric cues. In such cases, preserving precise spatial alignment through single-scale ROI-based fusion (SSF) is sufficient, while additional multi-scale semantic aggregation provides limited additional benefit.

In contrast, pedestrians and cyclists are thin, small, and often sparsely represented in LiDAR point clouds, especially at medium and long ranges. For these object categories, semantic information from camera images becomes crucial for resolving object presence, extent, and class identity. The proposed learnable FPN fusion (GMF and APF) consistently improves detection performance for pedestrians and cyclists, indicating that adaptively balancing image features across spatial scales is essential for robust small-object detection.

Compared to heavier multimodal architectures such as MVX-Net and transformer-based BEV fusion methods, FusionPointPillars offers a favorable balance between detection accuracy and computational efficiency. While approaches such as Multimodal PointPillars (M-PP) achieve higher pedestrian BEV accuracy, they rely on more aggressive point-wise fusion strategies and are evaluated on substantially more powerful hardware. FusionPointPillars, by contrast, preserves the simplicity and efficiency of the PointPillars framework while introducing a principled, end-to-end pillar-wise fusion mechanism that is compatible with resource-constrained deployment settings.

From our perspective, the main value of FusionPointPillars is not that it outperforms every existing multimodal detector on every metric, but that it improves difficult small-object cases without giving up the practical efficiency of PointPillars. This makes it a reasonable option when computational budget is limited and a full transformer-based fusion pipeline would be too expensive.

5.2 FUSION OF POINTPILLARS WITH BEVFORMER

This track was intended to answer a practical question: if LiDAR features are already available in BEV form, where should they be injected into a BEVFormer pipeline? Our experiments compared three cases on nuScenes: a camera-only Baseline, a decoder-side fusion variant (Decoder/V3), and the full encoder-decoder model (Enc_Dec).

Enc_Dec achieved the best headline accuracy, improving mAP from 0.201 to 0.251 and NDS from 0.219 to 0.255. Decoder (V3), which applies LiDAR fusion only before the decoder, remained close to the Baseline in runtime but did not provide the same overall accuracy gain. This suggests that decoder-side fusion alone is feasible and computationally attractive, but the largest detection improvement comes from combining decoder-side injection with encoder-side cross-modal interaction.

Class-wise results show that LiDAR fusion helps most for car, truck, bus, and barrier. These are classes for which geometric structure is relatively strong in LiDAR. Gains are smaller for bicycle and construction vehicle, which may reflect the limited training volume and the simplified PointPillars LiDAR representation used in this project.

The latency benchmark also reveals an important trade-off. On our local machine, the Baseline achieved 259.32 ms latency, Decoder (V3) achieved 269.31 ms, and Enc_Dec achieved 431.47 ms. In other words, the full two-stage fusion model gave the best detection accuracy, but the decoder-only variant stayed much closer to the camera-only baseline in runtime. This trade-off is central to how we interpret the feasibility of the model.

The main limitation of this track is that all reported results are from single runs on a local RTX 3060 system, and the absolute scores remain below those of larger published BEVFormer models trained with more compute and larger backbones. We therefore present these results as a feasibility study rather than a claim of state-of-the-art performance. Even so, the consistent improvement of Enc_Dec over the camera-only baseline shows that the proposed fusion direction is technically workable and worth further study.

6 CONCLUSION

In this work, we presented *FusionPointPillars*, a lightweight multimodal 3D object detection framework that integrates camera information into the PointPillars architecture through pillar-wise fusion. By projecting pillar centers onto the image plane and performing localized ROI-based feature sampling, our method enriches LiDAR pillar representations with semantic cues from camera images while preserving the efficiency and simplicity of BEV-based processing.

We further introduced a learnable multi-scale fusion strategy using feature pyramid networks, allowing the model to adaptively balance high-resolution spatial details and high-level semantic information across image feature scales. Experimental results on the KITTI benchmark demonstrate that this design yields consistent improvements for challenging object categories such as pedestrians and cyclists in both BEV and 3D detection, while maintaining competitive performance for cars. In addition to accuracy improvements, FusionPointPillars maintains a favorable trade-off between performance and computational cost. Inference speed measurements show that the proposed method remains suitable for real-time or near–real-time deployment on resource-constrained hardware, highlighting its practicality for autonomous driving systems.

Overall, this study demonstrates that pillar-level multimodal fusion provides an effective middle ground between LiDAR-only detectors and computationally intensive multimodal architectures.

The second track extends this study to a transformer-based setting. On nuScenes, the full Enc_Dec model improved the camera-only baseline by +0.050 mAP and +0.035 NDS, showing that PointPillars-derived LiDAR BEV features can be integrated effectively into a BEVFormer-style detector. At the same time, the latency benchmark showed a substantial runtime cost for full two-stage fusion, while the decoder-side variant remained much closer to the baseline in speed. We therefore interpret the BEVFormerFusion results as a proof of feasibility: the fusion mechanism works, but its final practicality depends on how accuracy and latency are balanced in future versions.

7 FUTURE WORK

7.1 FUSIONPOINTPILLARS

Testing on nuScenes datasets with multi-camera images. We plan to extend our FusionPointPillars framework to the nuScenes dataset, incorporating multi-camera image inputs to evaluate its effectiveness in large-scale, multi-view 3D perception scenarios. This extension is important because nuScenes provides a more diverse and challenging benchmark with 360° visual coverage, enabling the assessment of cross-view consistency and robustness under complex urban conditions. Evaluating on multi-camera data will further validate the scalability and generalization capability of the proposed fusion strategy beyond single-view settings.

7.2 FUSION OF POINTPILLARS WITH BEVFORMER

Longer training and convergence analysis. Longer training and controlled ablations remain important next steps for the BEVFormerFusion track. In particular, future work should compare the Baseline, Decoder (V3), and Enc_Dec variants under identical training budgets and, if possible, across multiple random seeds. This would clarify how much of the observed gain comes from the decoder-side fusion stage alone and how much additional benefit is provided by the encoder-side fusion stage.

A related next step is to improve convergence analysis under limited local compute. Because the current results are single-run results obtained on one RTX 3060 GPU, longer training and repeated runs would help determine whether the performance gap between Decoder (V3) and Enc_Dec is stable or partly reflects under-training.

Velocity estimation under LiDAR fusion. Since LiDAR-enriched BEV features partially dilute the TSA temporal signal, future work should explore (a) routing the velocity head to a snapshot taken before all LiDAR injection including encoder-side blending, (b) reducing the encoder-side blend weight for deeper encoder layers, and (c) explicit temporal LiDAR modeling through point-cloud-to-BEV temporal alignment.

Adaptive modality blending. The current per-layer scalar blend weight is spatially uniform. Query-dependent or spatially adaptive gating could allow the model to draw more heavily on LiDAR in regions with dense, reliable coverage and fall back to camera features in sparse or distant regions.

Missing-modality robustness. Neither model has been evaluated under sensor dropout. Dropout-based training or dedicated fallback pathways should be investigated for deployment scenarios where one modality is temporarily unavailable.

Calibration robustness. Both fusion stages rely on precise LiDAR-to-image calibration. Evaluating performance degradation under synthetic calibration perturbations and developing robust projection mechanisms would improve reliability.

Runtime and memory profiling. Our initial latency benchmark already shows that Enc_Dec is substantially slower than both the camera-only Baseline and Decoder (V3). Future work should extend this analysis with GPU-memory profiling, per-stage timing, and larger-scale deployment-oriented benchmarking.